

15/01/2026

Practical No: 1

Title :-

Perform tokenization (whitespace, punctuation based, treebank, tweet, MWE) using NLTK Library. Use the porter stemmer & snowball stemmer for stemming use any technique for lemmatization.

Objectives :-

- 1) To understand & implement different tokenization methods, whitespace, based MWE tokenization.
- 2) To compare porter stemmer & snowball stemmer.
- 3) To perform lemmatization using wordnet.
- 4) To analyze difference between stemming & lemmatization.

Course Outcome :-

- CO1 :- Understand fundamental text processing in NLP
CO2 :- Apply tokenization, stemming & lemmatization
CO3 :- Compare different NLP processing methods.

HW Requirements :-

- System :- Intel Core i3 or higher processor
RAM :- minimum 4GB
Hard disk :- minimum 20 GB
Display :- standard monitor with min. 1280 x 768

* Requirement :-

OS :- Windows 10/11, Linux

Python :- Version 3.7 or ↑

IDE :- Jupyter Notebook

Libraries :- NLTK 3.5 or high, Memory Padders.

Theory :-

1) Tokenization :- Tokenization is the process of breaking down text into smaller units called tokens. It is a foundation step in text processing.

* Types of Tokenization :-

a) whitespace Tokenization :- splits text based on whitespace characters (spaces, tabs, newlines). Simple but does not handle punctuation efficiently.

b) Punctuation based Tokenization :- separates word & punct marks into individual part trees punct & separate tokens.

c) Treebank Tokenization :- uses regular expression following penn treebank conventions. Handles convention (e.g., can't → c'd, n't)

d) Tweet Tokenization :- specialized for social media that preserves hashtags mentions, emojis. & handles repeated characters.

2] stemming :-
stemming reduces words to their root or base form by removing suffixes using lexical rules. It is a chopping process that may not always produce valid words.

a) Porter stemmer is the most widely used stemming only applies series of rules to strip common suffixes for e.g. running str.

b) snowball stemmer is also known as Porter2 stemmer. an improved & more aggressive version of Porter stemmer.

3] Lemmatization :-
Lemmatization reduces words to their dictionary form lemma using vocabulary & morphological analysis. Unlike stemming it always produces valid words & consider the context.

Algorithm	Procedures
Step 1:	Import necessary libraries.
Import	NLP Library
Import	Tokenization
Import	Stemmer
Import	Lemmatization

Step 2: Download NLTK

- Download 'punkt' for sentence tokenization
- Download 'wordnet' for lemmatization.
- Download 'Avg-Perceptron' for POS tagging.

Step 3: Perform Tokenization

- create text with various elements.
- initialize each tokenizer.
- Apply tokenization & applied result.

Step 4: Perform Stemming

- define list of wordnet in different forms.
- Initiates stemming initialize, apply stemming.

Step 5: Perform Lemmatization

- Define a list of word for lemmatization individual wordnet.
- apply lemmatization apply with specific position.
- compare with O/P.

Step 6: Analyse & compare Result.

- compare Tokenization, O/P. compare stemming vs. lemmatization.

* observations :-

Token method	sample I/p	Token count	Handle Punct	Handle #
whitespace	I'm learning NLP	3	No	No
Punctuation - Based	I'm learning NLP	6	yes	partially
Treebanks	I can't wait	5	yes	No
Tweet	# NLP @ user	1) 3	yes	yes
MWE	New York city	2	No	No
Technique	word running	word better		Valid Word
stemming	Run	better		Not always
lemmatization	Run	good		Always

conclusion :- The experiment demonstrates various NLTK text pre-processing methods. Different Tokenizers handle text differently based on context with specialized ones improving accuracy. Stemming is faster by white spaces and lemmatization.

Practical No-2

Topic :-

To implement & analyze text vectorization including bag of words, TF IDF & word2vec embeddings for converting text data into numerical expected.

Objective :-

- Implement Bow (Count & Normalized Count)
- Compute TF IDF & understand word imp.
- compare different text vectorization technique.
- Analyze word similarity using vectors.

H/W Requirement :-

- System Intel Core 3 or $\uparrow$
- RAM $\geq$ min 4 GB
- Hard disk $\geq$ min 20 GB
- Display - standard.

S/W Requirement :-

- OS $\geq$ windows 10/11, Linux
- Python : Version 3.7 or $\uparrow$
- IDE - Jupyter Notebook.
- Libraries $\geq$ sklearn 1.0, NLTK, Numpy, Pandas, matplotlib.

Theory is

- 1) Bag-of-words (BOW) model is
Bag of words text representation
technique that converts text to
numeric features vectors. Counting
word occurrences. It treats each
as an unordered collection of
words. ignoring grammar & word
order but maintaining word
frequency info.

word principle is

- create a vocabulary word from entire corpus
- for each document count occurrences of each vocabulary word.
- a) count occurrences
- b) Normalized count occurrences.

* adv

- 1) simple to understand.
- 2) works well for classification
- 3) computationally efficient.

2) TF IDF (Term Frequency Inverse Doc freq) is

Term freq. measures how frequency
a term in a document.

$$TF(t, d) = \frac{\text{No. of times of appears in a doc. d.}}{\text{Total No. of terms in doc. d.}}$$

Inverse doc frequency measures how imp a term is all documents.

$$IDF(t) = \frac{\text{Total No. of doc}}{\text{No. of doc containing term } t}$$

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

3] Word2vec embeddings →
word2vec is a neural N/W based technique that creates dense vector representation of words by learning from their context in large text corpora. Unlike Bow & TFIDF word2vec captures semantic relations between words.

Algorithm / Procedures →

- 1) Import required libraries
- 2) Prepare text corpus
- 3) Implement Bow
- 4) Implement Normalized count occurrences
- 5) Implement TFIDF.
- 6) Create word2vec embeddings.
- 7) Analyze & Compare result.

Observation →

Technique	Representation	Need in	Sparsity
Bow	sparse	high	high
Bow	sparse	high	high
TFIDF	sparse	high	high
Word2vec	sparse	high	Low

Conclusion :-

The experiment successfully implemented three major text vectorization techniques: BoW, TF-IDF, word2vec, and embeddings.

* Title :-

To perform Comprehensive text processing including text cleaning, lemmatization, stop words removal, label encoding & create TF IDF representation for text classification tasks.

* Objectives :-

- To clean raw text data by removing noise Punctuation, number & special character
- To apply lemmatization for reducing word forms.
- To remove stop word to focus.
- To encode categorical labels into numeric format.

* H/W requirements :-

System :- Intel Core I3 or more.

RAM :- min 4 GB

Hard disk :- 20 GB free

Display :- standard.

* S/W requirements :-

OS :- windows 10/11.

Python version 3.7

IDE :- Jupyter Notebook, Pycharm

Library :- scikit-learn, nltk, numpy

Theory :-

1) Text cleaning :-

- removes noise & irrelevant data from data
- Improves models performance.

* Common operations :-

- lowercasing (uniform text)
- Removing URL & emails.
- Removing HTML tags.
- Removing numbers
- Removing punctuation & special characters.
- white space normalization.

2) Lemmatization :-

- Converts words to their base form.
- produces meaningful words.
- Approach :-
 - dictionary based, rule based, wordnet based

EX :-

Running → Run, better → Good, Computers → Computer.

3) Stop words Removal :-

- Removes common words (a, the, is) with little
- Helps to focus on imp & reduce data size.

Methods :- i) NLTK stop word.

ii) Scikit-learn stop words.

4) Label encoding $\Rightarrow$

- converts categorical labels into numeric values.
- maintains consistent mapping.

* Features

- 1) Alphabetical arrangement
- 2) Reversible words.
- 3) more memory efficient to one-hot encoding.

5) TF - IDF

- measures importance of a word in a document

- formulas $\Rightarrow$

- i) TF - frequency of word in doc.
- ii) IDF - importance across all doc.
- iii) $TF \cdot IDF = TF \times IDF$.

Key points $\Rightarrow$

- High Score $\rightarrow$ imp. word
- low score $\rightarrow$ common word
- Create sparse matrix used in text & information retrieval.

* Algorithm $\Rightarrow$

- 1) Import libraries (numpy, pandas)
- 2) Load or create dataset.
- 3) apply preprocessing
- 4) Convert text using TF IDF
- 5) TF IDF Vectorization
- 6) Save o/p.

observations :-

Ser.No.	Category	encoded	level
1)	AIML	0	
2)	Data science	1	
3)	Deep Learning	2	
4)	M/C learning	3	
5)	NLP	4	
6)	programming	5	

Conclusion :-

The experiment successfully implemented a complete text preprocessing pipeline including text cleaning, lemmatization, stop words removal, label encoding & TF IDF representation.

Practical No - 4

24/02/2020

Title :-

To implement a Complete Transformer architecture from scratch using the Pytorch library. understanding all fundamental components including multihead attention, positional encodings, encoder, decoder, architecture feed forward N/w.

Objectives :-

- To understand of transformer architecture as proportion in "attention all you need" Paper.
- To implement I/p Embeddings & positional need for sequence processing.
- To build multihead self attention mechanism.

* H/W Requirement

- system : Intel core i3 or more.
- RAM :- min 4GB
- Hard disk : min 20 GB free
- Display : standard.

* S/W Requirements :-

- O.S. windows 10/11
- Python : version 3.7, or more.
- IDE :- Jupyter Notebook
- Libraries : Scikit-learn 1.0 or more. or NLTK.

* Theory *

- Transformers architecture *

1) overview *

The transformer is a deep learning model introduced in "Attention is all you need". It uses attention mechanism only (No RNN) for sequence modelling.

2) Input embeddings *

Converts tokens into dense vectors of size d_{model} . Scaled by vector database model to stabilize training.

3) Positional encoding *

- adds position info since transformers adds sequence order.

- uses $\sin$ & $\cos$ Helps model understand token position and generalize to longer sequence.

4) Multi-Headed Attention: *

- Allow model to focus on different parts of I/P simultaneously.

- Scaled dot product attention.

5) Feed forward network (FFN) *

- fully connected layer applied independently to each point.

- uses ReLU Activation function.

- Dimension: $\text{DFF} = 4 \times d_{\text{model}}$.

6) Layer Normalization is

- Normalize I/P across features
- Stabilizes & speeds up trainings.

7) Residuals Connection

- adds I/P & o/p of each sub layer.
- improves gradient flow.
- o/p = layer Norm.

8) Encoder - each layer includes

- multihead self attention
- feed forward NN
- residual connection
- layer Normalization.

9) Decoder - each decoder includes

- masked self attention (no future token)
- cross attention
- FFN, Residual connection + Normalization.

10) Complex Transformers

- I/P & o/p embedding + positional embeddings
- stack of Encoder & decoder layer.
- final linear layer & softmax for predict.

* Algorithm :-

- 1) Import libraries
- 2) I/P embeddings
- 3) Positional embeddings
- 4) Layer Normalization

- 5) Feed forward Network
- 6) Multihead attention
- 7) Residual Connection
- 8) Encoder Block
- 9) Encoder state
- 10) Decoder block.

* Observation :-

Sr.No.	components	Value	Description
1)	d-model	512	model embedding
2)	N (layers)		No. of layers
3)	h (heads)		No. of Attention heads
4)	d - FF		FF Hidden dimension
5)	d - k		Dimension per Attention
6)	Dropout	0.1	Drop probabilities

* Result / Conclusion :-

The experiment successfully implemented a complete transformer architecture from scratch using pytorch following the 'Attention is All you need' Paper. Has been used.

Practical - 5

19/03/2026

* Title is

To study and understand the concept of morphology in NLP through the implementation & analysis of add, delete & edit operations.

* Objectives is

- Understand morphology (word & format)
- Identify morphemes (small mean units).
- Implement add - delete operations.
- Analyze prefixes, suffixes & root.
- Differentiate inflectional & derivational morphology.

* H/W Requirements is

- Processor is intel i3 or more
- RAM is 4GB
- Storage is 20GB free
- Display - min 1024 X 768.

* S/W Requirements is

- OS is Windows 10.
- Python 3.7 or more.
- IDE is Jupyter Notebook
- Libraries is string, collections.

* Theory is

1) Morphology is

- Study of word structure format form small units.
- Helps to understand meaning &

Grammatical roles of words.

2) Morphemes $\rightarrow$

- Smallest meaningful units in language

* Types $\rightarrow$

- 1) Free morpheme - independent words.
- 2) Bound morpheme - Attached form (ending)
- 3) Root / base - core meaning.
- 4) Affixes - prefix, suffix, infix.

3) Types of morphology $\rightarrow$

- Inflectional $\rightarrow$ change grammar, not meaning.
- Derivational $\rightarrow$ create new words/class.

4) Add / Delete Table $\rightarrow$

- Tool to study word transformation using operation
- Add $\rightarrow$ insert char. at any position
operation = $(n+1) \times 26$.
- Delete $\rightarrow$ Remove character from any position
operation = n
Total operation = $(n+1) \times 26 + n$.

5) Morphological Analysis $\rightarrow$

- Breaks words into morphemes & identifies roles.
- Application $\rightarrow$ lemmatization, stemming, pos tagging, information retrieval, m/c translation, text to speech.

- 6) Morphological Parsing $\Rightarrow$
- Analyze words to identify morphemes
 - ~~#~~ Approach $\Rightarrow$
 - Rule-Based
 - Statistical
 - Hybrid.

- 7) Edit distance & morphology $\Rightarrow$
- measures minimum operations (add/delete) to convert words.
 - used - speech checking, word-similarity, morpheme generation.

Algorithm $\Rightarrow$

- 1) Define morphemes (root, prefix, suffixes)
- 2) Implement add operation (insert letter).
- 3) Implement delete operation (remove letter)
- 4) Generate add delete table (tabular format).
- 5) Identify morpheme (prefix, suffix, root)
- 6) Analyze transformation between words.
- 7) calculation of total operation.
- 8) Test with simple word & doc.
result.

* conclusion.

The experiment successfully demonstrated morphological analysis using add delete tables as a computational tool.